

Seesaw Programming Project

For this project, you'll be using a publicly available data source of schools in a CSV format. You'll download the data file and write two Python programs to manipulate the data.

We use Python for our backend code primarily at Seesaw, but we also don't want you to have to learn Python just for this project. If you'd be more comfortable completing this task using another language, that's OK -- just replace all references to "Python" or Python libraries below with the language of your choosing and equivalent libraries in that language.

Instructions

1. Please implement all features using pure python.
 - You will need the following modules: `csv`, `time`
 - You may use other modules in the python standard library, except for anything that will implement too much for you, e.g. `sqlite3`.
 - **If you're not sure whether a certain module is okay, please ask!**
 - **No other (non-stdlib) libraries may be used.**
2. Please provide a README specifically detailing how to run your code, including any installation requirements. Feel free to include anything additional you'd like us to understand, such as the thinking behind your design.
3. When you're done with both parts of this project, zip up your project directory and upload it to Google Drive. Make sure we have access and send us a link. Please include a snapshot of your running the code and its output.
4. We expect this to take 2-3 hours to complete (Parts 0 & 1 are intended for you to familiarize yourself with the data; most of your time should be spent on Part 2). If it seems like it will take significantly longer than that to complete, please ask us first to make sure you aren't over-complicating the project.

Part 0: Get the data

- Go to [this page](#) and download the dataset described as:
 - Year 2005-2006 (v.1b), States A-I, ZIP (769 KB) CSV File
This is the only file you will be using. You can ignore the other files on this page.
 - You may also find it helpful to consult the [documentation for this dataset](#).
 - Unzip the file, rename it to `school_data.csv`, and put it in the directory where you'll write your Python program.

- Now you have the data you need to get started. Note: the CSV file is in CP-1252 encoding.

Part 1: Load data from CSV and compute stats.

Write a method (or a method per question if you'd prefer) that loads the data out of the CSV file you downloaded and computes answers to the following questions:

1. How many total schools are in this data set?
2. How many schools are in each state?
3. How many schools are in each Metro-centric locale?
4. What city has the most schools in it? How many schools does it have in it?
5. How many unique cities have at least one school in it?

Guidelines

1. Create a file called `count_schools.py`, and write a method called `print_counts`.

Example output (this is just a sample, the lists have been truncated and counts aren't correct)

```
>>> count_schools.print_counts()  
Total Schools: 10000
```

```
Schools by State:
```

```
CO: 1000  
DC: 200  
AK: 600  
DE: 300  
AL: 1500  
AR: 1100  
...
```

```
Schools by Metro-centric locale:
```

```
1: 3000  
3: 2000  
2: 5000  
5: 300  
...
```

City with most schools: CHICAGO (50 schools)

Unique cities with at least one school: 1000

Part 2: Search over school data

We'd like teachers to be able to easily find the school they teach at. In order to do this, we'd like to offer a search feature that lets them search for their school using plain text.

1. This feature should search over school name, city name, and state name.
2. The top 3 matching results should be returned (see below for examples).

Guidelines

1. When a query doesn't match exactly, you'll need to come up with a set of rules to rank results. In particular, make sure more precise matches show up at the top of the list, and if there isn't an exact match, but there is a close match, some results are returned. There is no perfect set of rules, but you should come up with a set that improves the end user search experience as much as possible.
2. Searches should run in real-time, meaning that they should return results to the user in less than 100ms. It's ok to perform data loading and processing up front that takes longer than this if you'd like.
3. Create a file called `school_search.py`, and write a method called `search_schools`.

Evaluation

1. Accuracy
 - We'll evaluate the accuracy of your search using sample queries. We have included a few below in Test Cases that you can test with (we'll also test with others).
 - The following queries should return the results shown below as the top hits. If multiple results are shown below, it doesn't matter if one appears before the other, but they must be the first results returned.
 - If you see `[Next Best Hit]`, that means that you should include a reasonable hit for the search query, but that there isn't a specific hit that we'll be looking for.
2. Performance
 - All queries should return within 100ms.
 - **EXTRA CREDIT:** All queries are returned within 10ms.
3. Code Quality
 - We're looking for clear, easy to understand code. Write code that makes your thinking algorithm as obvious as possible.

- Consider standard production requirements like performance, readability, and maintainability.
- Don't worry about documentation for this project. Do your best to make the code readable without requiring reading lots of documentation.

FAQ

- I noticed X about the data, how should I handle that?
 - You may choose how to handle any data discrepancies. Be prepared to justify your decisions.
 - Your program should handle the data without errors.
- Do I need to handle typos/mis-spellings?
 - No.

Test Cases

elementary school highland park

```
>>> school_search.search_schools("elementary school highland park")
Results for "elementary school highland park" (search took: 0.009s)
1. HIGHLAND PARK ELEMENTARY SCHOOL
   MUSCLE SHOALS, AL
2. HIGHLAND PARK ELEMENTARY SCHOOL
   PUEBLO, CO
3. [Next Best Hit]
```

jefferson belleville

```
>>> school_search.search_schools("jefferson belleville")
Results for "jefferson belleville" (search took: 0.000s)
1. JEFFERSON ELEM SCHOOL
   BELLEVILLE, IL
2. [Next Best Hit]
3. [Next Best Hit]
```

riverside school 44

```
>>> school_search.search_schools("riverside school 44")
Results for "riverside school 44" (search took: 0.002s)
1. RIVERSIDE SCHOOL 44
   INDIANAPOLIS, IN
2. [Next Best Hit]
3. [Next Best Hit]
```

granada charter school

```
>>> school_search.search_schools("granada charter school")
Results for "granada charter school" (search took: 0.001s)
1. NORTH VALLEY CHARTER ACADEMY
   GRANADA HILLS, CA
2. GRANADA HILLS CHARTER HIGH
   GRANADA HILLS, CA
3. [Next Best Hit]
```

foley high alabama

```
>>> school_search.search_schools("foley high alabama")
Results for "foley high alabama" (search took: 0.001s)
1. FOLEY HIGH SCHOOL
   FOLEY, AL
2. [Next Best Hit]
3. [Next Best Hit]
```

KUSKOKWIM

```
>>> school_search.search_schools("KUSKOKWIM")
Results for "KUSKOKWIM" (search took: 0.001s)
1. TOP OF THE KUSKOKWIM SCHOOL
   NIKOLAI, AK
(No additional results should be returned)
```